

Group Member:

Angelina Jolie Candaya – 2802541644

Aqila Nailal Husna – 2802530565

Elizaveta Falisha Susanto – 2802530590

Fauzan Surya Yudistira – 2802544841

Hash Table

1. Background

In computer science, the efficient organization and retrieval of data are essential, especially as the volume of data increases rapidly. Traditional data structures such as arrays and linked lists often require linear time ($O(n)$) to search for an element, which becomes inefficient with large datasets. To address this issue, more advanced data structures like the hash table have been developed. A hash table uses a technique known as hashing, which converts a key into an index within an array, allowing for near-instant access to values. This makes hash tables a fundamental component in many computing applications, from database indexing to symbol tables in compilers.

2. Literature Review consists of theory of related/chosen data structure and general theory (*theory that align with the case of the program*).

1. Introduction to Hash Table

A hash table is a commonly used data structure to store an unordered set of items, allowing constant time inserts, lookups, and deletes (in expectation). Every item consists of a unique identifier called a key and a piece of information. For example, the key might be a Social Security Number, a driver's license number, or an employee ID number. The way in which a hash table stores an item depends only on its key, so we will only focus on the key here — but keep in mind that each key is usually associated with additional information that is also stored in the hash table.

2. Characteristics of Hash

- 1. Deterministic**

Hash function always produces the same output for the same input.

- 2. Non-Reversible**

Hash functions are designed to be one-way functions, meaning it is impossible to get the original input from the hash output.

- 3. Fixed Output Size**

Hash function always produces the same size of output regardless of the input size. For example, a common hash function called SHA-256 generates a 256-bit (32-byte) hash value.

- 4. Uniform Distribution**

A good hash function evenly distributes hash values across its output space, reducing the likelihood of collisions.

- 5. Avalanche Effect**

A small change in the input should result in a significant change in the output, making it difficult to predict the original input based on the hash value alone.

3. Operations in Hash Table Data Structure

A hash table supports the following operations, each operation will take constant time (in expectation):

- 1. Search**

Used to find an element/data in the hash table based on its key or index. The search operation uses a hash function to generate the index of the element being searched.

- 2. Insertion**

This operation is used to insert new elements/data into the hash table. The new element is placed at the index generated by the hash function.

- 3. Deletion**

Used to remove an element/data from the hash table based on its key or index. The element is removed from the index generated by the hash function.

- 4. Update**

This operation is used to modify the value of an existing element/data in the hash table. The value can be updated based on the key or index.

- 5. Collision Handling**

A collision occurs when two or more elements are assigned the same index by the hash function. This operation ensures that elements with the same index can still be stored and accessed correctly.

6. Resize

This operation is used to change the size of the hash table when the number of elements/data exceeds the predefined capacity. Resizing helps maintain the efficiency and performance of the hash table.

7. Iteration

This operation is used to access and process all the elements/data in the hash table sequentially.

4. Hashing

Hashing is a technique used to map data keys to specific indices in a hash table using a hash function. This function transforms a key, such as a string or number, into an index where the associated data will be stored. For example, a simple hash function may extract the first character of a string and convert it into a number.

The goal of hashing is to allow efficient data retrieval, ideally in constant time, which makes it highly suitable for applications requiring frequent access, like symbol tables, caches, and indexing systems.

5. Hash Function

To determine the exact location in the table where the data will be stored, the key k is processed using a function called the hash function, denoted as $h(k)$. The function $h(k)$ maps the input key k to a specific integer value known as the hash index, which corresponds to a position in the hash table's internal array. This transformation enables fast and direct access to the data associated with each key. The keys should be evenly distributed across the array via a decent hash function to reduce collisions and ensure quick lookup speeds. Selecting a decent hash function is based on the properties of the keys and the intended functionality of the hash table. Using a function that evenly distributes the keys and reduces collisions is crucial.

1. Criteria Based on Which a Hash Function Is Chosen

- a. To ensure that the number of collisions is kept to a minimum, a good hash function should distribute the keys throughout the hash table in a uniform manner. This implies that for all pairings of keys, the likelihood of two keys hashing to the same position in the table should be rather constant.

- b. To enable speedy hashing and key retrieval, the hash function should be computationally efficient.
- c. It ought to be challenging to deduce the key from its hash value. As a result, attempts to guess the key using the hash value are less likely to succeed.
- d. A hash function should be flexible enough to adjust as the data being hashed changes. For instance, the hash function needs to continue to perform properly if the keys being hashed change in size or format.

2. Common Hash Function Techniques

There are many ways to map key into an indeks. The following are some of the important methods for constructing hash indeks:

1. Mid-square Hash Function

The mid-square method is a classical hashing technique where the key is squared and a portion of the resulting value—typically the middle digits or bits—is extracted to form the hash index. This method allows all parts of the key to influence the result, increasing the likelihood of a more uniform distribution even for similar keys.

To apply this method, the key is first squared. Then, a fixed number of bits or digits from the middle of the squared result are selected to determine the final index. For string keys, a numeric conversion is done beforehand. This technique is simple yet effective, especially when the table size is a power of two. The middle part can be extracted efficiently using bit masking and shifting operations. Its main advantage lies in reducing clustering by spreading hash values more evenly across the table.

➤ Formula:

$$h(k) = \text{middle } r \text{ digits of } (k^2)$$

➤ Example:

Let $k = 3121$

$$k^2 = 9740641$$

Extract middle 3 digits = 406

$$\text{So, } h(3121) = 406$$

2. Division Hash Function

The division method is the simplest and most widely used hash function, where the key is divided by a fixed number (usually a prime), and the remainder is taken

as the hash index. This method ensures that keys are distributed across the table with minimal computational effort.

➤ Formula:

$$h(k) = k \bmod M$$

➤ Where:

k is key.

M is the table size or a chosen prime number.

➤ Example:

If the key is a string, such as "ABCD", we can compute its ASCII total:

$$65+66+67+68=266$$

Then apply modulo with table size 97:

$$h("ABCD") = 266 \bmod 97 = 72$$

Thus, the string "ABCD" is stored at index 72 in the hash table.

3. Folding Hash Function

The folding method is a hashing technique where the key is divided into smaller parts of equal length, and these parts are added together to generate the final hash value. It is especially useful when the key has many digits. This method is effective in producing a reasonable distribution of hash values, even when keys are similar in structure.

➤ Steps:

1. Divide the key into equal parts (e.g., 2-digit segments).
2. Add all parts together.
3. If the result exceeds the table size, take the last two digits (i.e., ignore the carry).

➤ Example:

Let key = 5678.

Divide into: 56 and 78 -> Sum = 134.

Hash index = 34 (last two digits of 134).

4. Digit Extraction Hash Function

Digit extraction is a simple and quick hashing method. The idea is to pick specific digits from a number (the key) and use them to create the hash value. The digit positions are usually chosen beforehand, like taking the 1st, 3rd, and 5th digits of the number. This method works best when the selected digit positions are

diverse and well-distributed across the key range. However, it may lead to clustering if the same digit positions contain low variability.

➤ Example:

Let's say the key is $x = 14568$.

Extract the 1st, 3rd, and 5th digits, which are 1, 5, and 8.

Then the hash value is 158.

5. Rotating Hash Function

Rotating hash is a method that starts by applying a basic hash technique, such as division or mid-square. Once the initial hash address is obtained, the digits are rotated or shifted to form a new hash address. This extra step helps improve data distribution in the table and reduces the chances of collisions.

➤ Example:

Given a hash result of 20021, rotating the digits produces 12002. This is done by shifting the digits or folding them into a new sequence. This method adds variation to the hash value, making it more effective for handling similar input values.

6. Collision

Collisions happen when two or more keys point to the same array index.

There are two general ways to handle collisions:

1. Open Addressing

Collisions are handled by looking for the following empty space in the table. If the first slot is already taken, the hash function is applied to the subsequent slots until one is left empty.

There are various ways to use this approach, including linear probing and rehashing

b. Linear Probing:

When a collision occurs (i.e., two keys hash to the same index), linear probing searches for the next available slot by checking the subsequent indices one by one in a linear fashion.

c. Rehashing:

When the hash table is nearly full, the number of collisions increases, which reduces the performance of insertion and search operations when using linear probing. Linear probing may fail to find an empty slot because most or all slots are already occupied. In such cases, a better option is to create a new hash table with double the size of the

original table. All entries in the original hash table must then be rehashed and transferred to the new table.

2. Chaining

Chaining is a method used to handle collisions in a hash table. When two or more keys map to the same index, all the values are stored in a linked list at that index. Instead of replacing the existing value, new entries are simply added to the chain. This approach avoids overwriting data and keeps all entries accessible. Searching through the chain takes linear time in the worst case, but with a good hash function, chains remain short and efficient.

➤ Example:

If keys 15 and 50 both hash to index 0, both will be linked at that index:

Index 0 -> 15 -> 50.

7. Applications of Hash Table:

1. Hash tables are frequently used for indexing and searching massive volumes of data. A search engine might use a hash table to store the web pages that it has indexed.
2. Data is usually cached in memory via hash tables, enabling rapid access to frequently used information.
3. Hash functions are frequently used in cryptography to create digital signatures, validate data, and guarantee data integrity.
4. Hash tables can be used for implementing database indexes, enabling fast access to data based on key values.

8. Advantages and Disadvantages

1. Advantages of Hash Table

a. Dynamic Size

Hash tables can dynamically resize themselves as the number of elements changes.

b. Flexibility in Keys

Hash tables allow for flexible key types. Unlike arrays, where the keys are typically integers representing indices, hash tables can use a wide range of data types as keys, including strings.

c. Efficient Search, Insertion, and Deletion

Searching, Adding, and removing elements from a hash table is generally fast. The time complexity for searches, insertions, and deletions in a hash table is also typically $O(1)$ on average.

2. Disadvantages of Hash Table

a. Unordered Data

Hash tables do not guarantee an order of elements. If you need to maintain a specific order, arrays may be more suitable since they store elements contiguously.

b. Additional Overhead

Hash tables have additional memory overhead due to the need for hash functions, hash buckets, and potential collisions.

References:

Fakultas Ilmu Komputer dan Teknologi Informasi (FIKTI) UMSU. (n.d.). *Struktur Data Hash Table: Pengertian, Cara Kerja, dan Operasi Hash Table*. Universitas Muhammadiyah Sumatera Utara.

<https://fikti.umsu.ac.id/struktur-data-hash-table-pengertian-cara-kerja-dan-operasi-hash-table/>

Fukunishi, M. (2021, May 6). *Understanding Hash Tables: Features, Pros, Cons, Time Complexity and Implementation*. DEV Community. <https://dev.to/masakifukunishi/understanding-hash-tables-features-pros-cons-time-complexity-and-implementation-2nm>

<https://dev.to/masakifukunishi/understanding-hash-tables-features-pros-cons-time-complexity-and-implementation-2nm>

GeeksforGeeks. (n.d.). *Hash Table – Data Structure*. <https://www.geeksforgeeks.org/hash-table-data-structure/>

Stanford University. (2018). *CS161 Lecture 8: Hashing* [PDF].

<https://web.stanford.edu/class/archive/cs/cs161/cs161.1182/Lectures/Lecture8/CS161Lecture08.pdf>

3. Benefits

- a. **Fast Data Access:** Hash tables provide average-case constant time complexity ($O(1)$) for insertion, deletion, and retrieval, making them highly efficient.
- b. **Efficient Memory Use:** With appropriate hash functions and collision resolution, hash tables can manage memory usage effectively.
- c. **Key-Based Storage:** Data is stored in key-value pairs, which simplifies data lookup and manipulation.
- d. **Widely Applicable:** Hash tables are widely used in real-world applications such as caches, databases, and language interpreters due to their speed and simplicity.
- e. **Improves Algorithm Performance:** Many algorithms that require frequent lookups benefit significantly in performance when hash tables are used.

4. Result

- Screenshots from the program's features:

a. Insert

```
Menu:
1. Insert
2. Search
3. Display
4. Exit
>> 1
Berapa banyak buku yang ingin dimasukkan: 8

Masukkan Judul buku: To Kill a Mockingbird
Masukkan Stock: 10
[!] Input berhasil

Masukkan Judul buku: 1984
Masukkan Stock: 20
[!] Input berhasil

Masukkan Judul buku: The Catcher in the Rye
Masukkan Stock: 30
[!] Input berhasil

Masukkan Judul buku: The Alchemist
Masukkan Stock: 40
[!] Input berhasil

Masukkan Judul buku: Harry Potter and the Sorcerer's Stone
Masukkan Stock: 50
[!] Input berhasil

Masukkan Judul buku: Pride and Prejudice
Masukkan Stock: 60
[!] Input berhasil

Masukkan Judul buku: The Little Prince
Masukkan Stock: 70
[!] Input berhasil

Masukkan Judul buku: The Fault in Our Stars
Masukkan Stock: 80
[!] Input berhasil

Tekan ENTER untuk melanjutkan...|
```

b. Search

```
Menu:
1. Insert
2. Search
3. Display
4. Exit
>> 2
Masukkan Judul buku yang dicari: 1984
Buku "1984" ditemukan dengan stock: 20

Tekan ENTER untuk melanjutkan...|
```

c. Display

```
Menu:
1. Insert
2. Search
3. Display
4. Exit
>> 3
Buku yang tersedia
[0]: | The Little Prince: 70 | --> | Pride and Prejudice: 60 | --> | To Kill a Mockingbird: 10 | --> NULL
[1]: | The Fault in Our Stars: 80 | --> NULL
[2]: | Harry Potter and the Sorcerer's Stone: 50 | --> NULL
[3]: | The Alchemist: 40 | --> NULL
[4]: | The Catcher in the Rye: 30 | --> NULL
[5]: | 1984: 20 | --> NULL

Tekan ENTER untuk melanjutkan...|
```

■ Program code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 10

void menu(){
    system("cls");
    puts("Menu:");
    puts("1. Insert");
    puts("2. Search");
    puts("3. Display");
    puts("4. Exit");
    printf(">> ");
}

void ENTER(){
    printf("\nTekan ENTER untuk melanjutkan...");
    getchar();
}

struct Node{
    char key[50];
    char value[100];
    struct Node* next;
};

Node* hashTable[TABLE_SIZE];

int hash(const char* key){
    int total=0;
    for(int i=0;key[i]!='\0';i++){
        total+=key[i]*(i + 1);
    }

    long square = (long)total * total; // pangkat 2
```

```

    char str[20];
    sprintf(str, "%ld", square);

    int len=strlen(str);
    int mid=len/2;

    int midVal=(len>=2) ? (str[mid-1]-'0')*10+(str[mid]-'0') : (str[0]-'0');

    return midVal % TABLE_SIZE;
}

void insert(const char* key, const char* value){
    int index=hash(key);
    Node* newNode=(Node*)malloc(sizeof(Node));
    strcpy(newNode->key,key);
    strcpy(newNode->value,value);
    newNode->next=hashTable[index];
    hashTable[index]=newNode;
}

void search(const char* key) {
    int index=hash(key);
    Node* current=hashTable[index];
    while(current){
        if(strcmp(current->key,key)==0){
            printf("Buku \"%s\" ditemukan dengan stock: %s\n",key,current->value);
            return;
        }
        current = current->next;
    }
    printf("Buku \"%s\" tidak ditemukan.\n",key);
}

void display(){
    puts("Buku yang tersedia");
    for (int i=0;i<TABLE_SIZE;i++){
        Node* curr=hashTable[i];
        if(curr){
            printf("[%d]: ",i);
            while(curr){
                printf("| %s : %s | --> ",curr->key,curr->value);
                curr=curr->next;
            }
            printf("NULL\n");
        }
    }
}

void inputData(){
    int banyakbuku;

    printf("Berapa banyak buku yang ingin dimasukkan: ");
    scanf("%d",&banyakbuku);getchar();
    puts("");

    for(int i=0;i<banyakbuku;i++){
        char key[50], value[100];

        printf("Masukkan Judul buku: ");
        scanf("%s",key);getchar();

        printf("Masukkan Stock: ");

```

```

        scanf("%[^\\n]", value); getchar();

        insert(key, value);
        puts("[!] Input berhasil");
        puts("");
    }
}

int main() {
    int pilihan;
    char key[50], value[100];
    do {
        menu();
        scanf("%d", &pilihan);
        getchar();
        switch (pilihan) {
            case 1:
                inputData();
                ENTER();
                break;
            case 2:
                printf("Masukkan Judul buku yang dicari: ");
                scanf("%[^\\n]", key); getchar();
                search(key);
                ENTER();
                break;
            case 3:
                display();
                ENTER();
                break;
            case 4:
                break;
            default:
                printf("Input tidak valid.\\nCoba lagi...");
                getchar();
        }

        }while(pilihan!=4);
    return 0;
}

```

- Program Documentation (the code/function with human language)

- a. **void menu()**

This function is responsible to display menu. It clears the screen with (system("cls")) and displays a list of options for the user:

Insert: To add new data, in this case, to add new books.

Search: To find data. In this case, to find a book.

Display: To show all data which are books in the hash table.

Exit: To terminate the program.

- b. **void ENTER()**

This is a function to pause the program's execution. It prints a message asking the user to press ENTER and waits for keyboard input before continuing. This allows people to read the output before the screen clears or the menu is shown again.

c. struct Node

This defines the structure for a node in hash table chaining. Each Node represents a book data and contains:

key[50]: A character array to store the book's title.

value[100]: A character array to store the book's stock information.

next: A pointer to the next Node in the list, used for chaining in case of hash collisions.

d. Node* hashTable[TABLE_SIZE]

This is the global declaration of the hash table itself. It is an array of pointers to Node. TABLE_SIZE is defined as 10, meaning the hash table has 10 slots (from index 0 to 9). Each slot can either be NULL (empty) if there is no data or point to the head of a linked list of Nodes.

e. int hash(const char* key)

This function calculates a hash index from a given string key (the book title) using the mid-square method. Operations that happen in this function such as:

- It converts the input string key into an integer total by summing the ASCII values of its characters, with each character's value multiplied by its position ($i + 1$).
- It squares this total.
- It converts the squared result into a string.
- It extracts the middle two digits from this string representation of the square. If the squared number has fewer than two digits, it takes the single digit.
- Finally, it returns this middle value modulo TABLE_SIZE to ensure the index is within the valid bounds of the hash table array.

f. void insert(const char* key, const char* value)

This function adds a new key-value pair (book title and stock) to the hash table.

- It first calculates the index for the given key by calling the hash() function.
- It dynamically allocates memory for a newNode.
- It copies the provided key and value into the newNode.
- It sets the newNode's next pointer to the current head of the linked list at hashTable[index].

- It then updates `hashTable[index]` to point to the `newNode`, effectively inserting the new node at the beginning of the list for that index.

g. `void search(const char* key)`

This function looks for a specific key in the hash table.

- It calculates the index for the key using the `hash()` function to know where to start searching.
- It traverses the linked list at that index, starting from `hashTable[index]`.
- In the loop, it compares the key of the current node with the key being searched for using `strcmp()`.
- If a match is found, it prints a success message with the book's stock and returns.
- If the entire list is traversed without finding the key, it prints a "not found" message.

h. `void display()`

This function prints the entire contents of the hash table.

- It iterates through each slot of the `hashTable` array from index 0 to `TABLE_SIZE - 1`.
- If a slot is not empty (`if(curr)`), it prints the index.
- It then traverses the linked list at that index, printing the key-value pair of each node in the chain until it reaches the end (`NULL`).

i. `void inputData()`

This function handles the user input for inserting one or more books.

- It asks the user how many books they want to enter.
- It then loops that many times.
- In each iteration, it prompts the user to enter the book's title (key) and its stock (value).
- It calls the `insert()` function to add the new book to the hash table and prints a confirmation message.

j. `int main()`

The operations in main function are:

- It declares variables for the user's menu choice (`pilihan`) and to temporarily hold key/value inputs.

- It enters a do-while loop that continues until the user chooses to exit (option 4).
- Inside the loop, it calls menu() to display the options and reads the user's choice.
- A switch statement then calls the appropriate function based on the user's input (inputData for case 1, search for case 2, display for case 3).
- The loop terminates when pilihan is 4. return 0; indicates successful program execution.

LINK:

Presentation video:

https://drive.google.com/file/d/1ZzoBXLNJ0my0mruezziGMH1QwQ-BeSeW/view?usp=drive_link

PPT:

https://www.canva.com/design/DAGf5XMOap8/QGGJFMn06w8agJCDjeiqBQ/edit?utm_content=DAGf5XMOap8&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton